

Reviewer Pack — Minimal Index of Formal Language Automorphism Groups and SAT...

Entry aa8b8c273058 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-12 06:57:14 UTC

Minimal Index of Formal Language Automorphism Groups and SAT Proof Length

Entry ID: aa8b8c273058 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-12 06:57:14 UTC

1. Conjecture

Field A (mathematical branch): Formal Language Theory - Automata Theory **Field B** (complexity object): Boolean Satisfiability Problem - SAT Proof Length

Statement:

For every satisfiable Boolean formula φ , the minimal index of its automorphism group is linearly correlated with its proof length in a standard proof system, such that $|\text{Aut}(\varphi)| = \Theta(w(\varphi))$ for any polynomial-time proof system.

Rationale (proposer's reasoning):

Automorphism groups capture symmetries of formal languages, which could potentially expose underlying structural properties that influence proof complexity. If the minimal index is correlated with proof length, it may indicate a connection between language structure and the difficulty of proving its satisfiability.

Taxonomy category: resolution_proof_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 2504cf7525b6d23a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 random seeds, the correlation coefficient between the minimal index of the automorphism group and the proof length exceeds 0.9 and no seed yields an absolute difference greater than 5 in the ratio of $|\text{Aut}(\varphi)|$ to $w(\varphi)$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - automorphism groups formal language AND SAT proof length - formal language theory automata theory intersection with Boolean satisfiability problem - Boolean formula automorphism group index related to proof length standard proof system

Top relevant hits considered: - [http://arxiv.org/abs/1511.09051v3] On automorphism groups of affine surfaces - [http://arxiv.org/abs/2104.14760v4] Automorphism and outer automorphism groups of Right-angled Artin groups are not relatively hyperbolic - [http://arxiv.org/abs/1010.3612v2] Explicit examples of equivalence relations and factors with prescribed fundamental group and outer automorphism group - [http://arxiv.org/abs/hep-th/9707234v2] Variational Approach to Quantum Field Theory: Gaussian Approximation and the Perturbative Expansion around It - [http://arxiv.org/abs/chao-dyn/9311011v2] The Great Inequality In A Hamiltonian Planetary Theory - [http://arxiv.org/abs/2605.30389v1] The Inclusion Depth of Pattern Languages: An Open Problem in Algorithmic Learning Theory - [http://arxiv.org/abs/1405.2700v1] Zero Excess and Minimal Length in Finite Coxeter Groups - [http://arxiv.org/abs/1511.06778v1] The 1st Fermi Lat Supernova Remnant Catalog

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_formula(n):
        if n == 1:
            return "x"
        else:
            subformulas = [generate_formula(random.randint(1, n-1)) for _ in range(2)]
            return f"({subformulas[0]} & {subformulas[1]})"

    def is_satisfiable(formula):
        if formula == "x":
            return True
        elif formula.startswith("(") and formula.endswith(")"):
            subformula = formula[1:-1]
            if subformula.startswith("(") and subformula.endswith(")"):
                left, right = subformula.split("&")
                return is_satisfiable(left) and is_satisfiable(right)
            else:
```

```

        return False
    else:
        return False

def compute_automorphism_group(formula):
    if formula == "x":
        return {0}
    elif formula.startswith("(") and formula.endswith(")"):
        subformula = formula[1:-1]
        if subformula.startswith("(") and subformula.endswith(")"):
            left, right = subformula.split("&")
            automorphisms_left = compute_automorphism_group(left)
            automorphisms_right = compute_automorphism_group(right)
            return {0}
        else:
            return set()
    else:
        return set()

def proof_length(formula):
    if formula == "x":
        return 1
    elif formula.startswith("(") and formula.endswith(")"):
        subformula = formula[1:-1]
        if subformula.startswith("(") and subformula.endswith(")"):
            left, right = subformula.split("&")
            return proof_length(left) + proof_length(right)
        else:
            return 0
    else:
        return 0

n_max = 40
instances_tested = 30
metric_values = []

for _ in range(instances_tested):
    formula = generate_formula(n_max)
    if is_satisfiable(formula):
        automorphism_group = compute_automorphism_group(formula)
        proof_len = proof_length(formula)
        metric_values.append((len(automorphism_group), proof_len))

if not metric_values:
    return {
        "metric_name": "proof_length",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": n_max,
        "conjecture_holds": False,
        "counterexample": "unsatisfiable_formula"
    }

mean_aut = sum(x[0] for x in metric_values) / len(metric_values)
mean_len = sum(x[1] for x in metric_values) / len(metric_values)
max_ratio = max(abs(x[0] / x[1]) for x in metric_values if x[1] != 0)

```

```

return {
    "metric_name": "proof_length",
    "metric_value": mean_aut,
    "instances_tested": instances_tested,
    "n_max": n_max,
    "conjecture_holds": max_ratio <= 5 and len(metric_values) > 10,
    "counterexample": "" if max_ratio <= 5 else f"max_ratio={max_ratio}"
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    mean_aut = sum(r["metric_value"] for r in results if r["conjecture_holds"]) / len(results)
    std_dev = math.sqrt(sum((r["metric_value"] - mean_aut) ** 2 for r in results if r["conjecture_holds"]) / len(results))
    support_fraction = sum(1 for r in results if r["conjecture_holds"]) / len(results)

    if all(r["conjecture_holds"] for r in results):
        print(f"RESULT: SUPPORTED mean={mean_aut} std={std_dev} support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next((r["seed"] for r in results if not r["conjecture_holds"]), None)
        print(f"RESULT: FALSIFIED counterexample='max_ratio_exceeded' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no_seeds_support")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b702afb3.py", line 110, in <module>
    result = run_trial(seed)
              ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b702afb3.py", line 75, in run_trial
    if is_satisfiable(formula):
        ~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_b702afb3.py", line 34, in is_satisfiable
    left, right = subformula.split("&")
    ~~~~~
ValueError: too many values to unpack (expected 2)

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which means it did not complete its execution to provide the necessary correlation coefficients and ratios for evaluation. | next: Re-run the test with proper error handling to ensure that it completes without crashing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	17409
2	propose	ollama_remote	glm4:latest	0	0	13217
3	preregistration	ollama_remote	glm4:latest	0	0	10143
4	novelty	ollama_remote	glm4:latest	0	0	9831
5	novelty	ollama_remote	glm4:latest	0	0	9799
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	33974
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14532
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12472
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12341
10	judge	ollama_remote	glm4:latest	0	0	43695

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 177414 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in *research/audit/aa8b8c273058.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/aa8b8c273058.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/aa8b8c273058.tar.gz* (if generated)